

Quectel EC200U-CN LTE Dongle on macOS

Complete Step-by-Step Setup Guide

Native USB internet connection — no vendor driver, no VM required

Techiesms · techiesms.com

This guide documents the full working process to detect, connect, and use a **Quectel EC200U-CN** LTE module as an internet connection on macOS, without any vendor driver or a Linux VM. It was built and verified end-to-end on macOS with a native `en` Ethernet-over-USB (ECM) interface.

Module identity confirmed during setup:

- USB Vendor ID: `0x2c7c` (Quectel)
- USB Product ID: `0x0901` (EC200U normal operating mode — AT + AP + CP + NMEA + DIAG + MOS interfaces)
- USB mode: **ECM** (`AT+QCFG="usbnet" = 1`) — this is why macOS can bind it natively as a network port, unlike RNDIS or QMI/MBIM modules which need vendor drivers macOS doesn't have.

Why this works without a driver

Quectel officially supports macOS drivers for **none** of their modules — official driver support is Windows, Linux, and Android only. However, the EC200U's **ECM (Ethernet Control Model)** USB mode is a USB-IF standard class that macOS has built-in support for. So instead of fighting for a driver, this guide uses:

1. macOS's **native ECM driver** to get a network interface (`enX`)
2. **Raw USB (libusb/pyusb)** to send AT commands directly to the module's vendor-specific AT interface, bypassing the need for a serial driver

Prerequisites

- A Mac with the Quectel EC200U-CN connected via USB
- [Homebrew](#) installed
- Terminal access with `sudo` privileges
- Your SIM's APN (find from your carrier if unknown — e.g. Jio: `jionet` , Airtel: `airtelgprs.com` , Vi: `www`)

Step 1 — Confirm the module is detected at the USB level

```
system_profiler SPUSBDataType | grep -i -B2 -A15 "2c7c"
```

You should see an entry like:

```
Product ID: 0x0901
Vendor ID: 0x2c7c
Manufacturer: Android
```

Note: The module reports itself as `Manufacturer: Android` — this is normal for Quectel modules and not an error.

If nothing appears here, it's a hardware/power/cable issue, not a driver issue — raw USB enumeration happens before any driver involvement. Try a different USB cable/port before proceeding.

Step 2 — Confirm macOS bound a native network interface

```
networksetup -listallhardwareports
```

Look for a new hardware port (often labeled "**USB Ethernet**" or similar) with a device name like `en10` (the exact number will vary per Mac).

```
ifconfig en10
```

At this stage it will show `status: inactive` — the interface exists at the driver level, but the modem hasn't been told to start a data session yet. That's expected; the next steps fix this.

Step 3 — Install the tools to talk to the module directly

```
brew install libusb
pip3 install pyusb --break-system-packages
```

Confirm Python can see the device:

```
python3 -c "
import usb.core
dev = usb.core.find(idVendor=0x2c7c, idProduct=0x0901)
print(dev)
"
```

This should print the full USB device descriptor, including 9 interfaces. Interfaces **2, 7, and 8** are AT-command-capable (they mirror each other); interfaces 5 and 6 are binary diagnostic/log ports; interfaces 3 and 4 do not respond to plain AT commands over raw bulk transfer.

Step 4 — The connection script

Save this as `ec200u.py`. It reboots the module (this clears any stale internal binding state — found to be essential during testing), waits for it to re-enumerate, then runs the SIM/network checks and binds the data call to the USB network interface.

```
import usb.core
import usb.util
import sys
import time

VID, PID = 0x2c7c, 0x0901
INTERFACE = 2 # usbat / AT command port
EP_OUT = 0x02
EP_IN = 0x84

def get_device():
    dev = usb.core.find(idVendor=VID, idProduct=PID)
    if dev is None:
        return None
    try:
        dev.set_configuration()
    except usb.core.USBError:
        pass
    try:
        if dev.is_kernel_driver_active(INTERFACE):
            dev.detach_kernel_driver(INTERFACE)
    except (usb.core.USBError, NotImplementedError):
        pass
    usb.util.claim_interface(dev, INTERFACE)
    return dev

def send_at(dev, cmd, wait=1.0, reads=3):
    print(f">> {cmd}")
    try:
        dev.write(EP_OUT, (cmd + "\r\n").encode())
    except usb.core.USBError as e:
        print(f"<< write failed: {e}")
        return ""
    time.sleep(wait)
    full = ""
    for _ in range(reads):
        try:
            data = dev.read(EP_IN, 512, timeout=1500)
            full += bytes(data).decode(errors="ignore")
        except usb.core.USBError:
            break
    print(f"<< {full}")
    return full

def wait_for_device(timeout=60):
```

```

print("\nWaiting for module to re-enumerate on USB...")
start = time.time()
while time.time() - start < timeout:
    dev = usb.core.find(idVendor=VID, idProduct=PID)
    if dev is not None:
        print("Device found again.")
        time.sleep(3)
        return True
    time.sleep(1)
print(".", end="", flush=True)
print("\nTimed out waiting for device to reappear.")
return False

def connect(apn="www"):
    # Step 1: connect and force a full reboot to clear stale state
    dev = get_device()
    if dev is None:
        sys.exit("Device not found")

    print("=== Rebooting module (AT+CFUN=1,1) ===")
    send_at(dev, "AT+CFUN=1,1", wait=3.0)

    try:
        usb.util.release_interface(dev, INTERFACE)
    except Exception:
        pass

    # Step 2: wait for it to come back
    if not wait_for_device():
        sys.exit("Module did not come back – check power/connection manually.")

    # Step 3: reconnect and run the full connect sequence
    dev = get_device()
    if dev is None:
        sys.exit("Could not reconnect to device after reboot")

    print("\n=== Checking SIM and network status ===")
    send_at(dev, "AT+CPIN?")
    send_at(dev, "AT+CSQ")
    send_at(dev, "AT+CGATT?")

    print("\n=== Binding data call to USB network adapter ===")
    send_at(dev, "AT+QNETDEVCTL=1,1,1", wait=3.0)
    status = send_at(dev, "AT+QNETDEVCTL?")

    try:
        usb.util.release_interface(dev, INTERFACE)
    except Exception:
        pass

    if "1,1,1,1" in status.replace(" ", ""):
        print("\n Data call bound successfully. Now run on the Mac side:")
        print("  sudo ipconfig set en10 DHCP  (replace en10 with your interface)")
    else:
        print("\n⚠ Bind may not have succeeded – check +QNETDEVCTL? output above.")

def disconnect():
    dev = get_device()
    if dev is None:
        sys.exit("Device not found")
    print("=== Unbinding data call ===")
    send_at(dev, "AT+QNETDEVCTL=0", wait=2.0)
    try:
        usb.util.release_interface(dev, INTERFACE)
    except Exception:
        pass

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "disconnect":
        disconnect()
    else:
        connect()

```

Run it:

```
sudo python3 ec200u.py
```

Expected final output includes:

```
+QNETDEVCTL: 1,1,1,1
✓ Data call bound successfully.
```

If the APN isn't already configured on the SIM (check with `AT+CGDCONT?` inside the script), add these two calls before the `QNETDEVCTL` bind step, replacing `"www"` with your carrier's APN: `python send_at(dev, 'AT+QICSGP=1,1,"www","","",1') send_at(dev, "AT+QIACT=1", wait=5.0)`

Step 5 — Bring up the IP on the Mac side

Find your interface name (it may differ from `en10` on another Mac):

```
networksetup -listallhardwareports
```

Then request a DHCP lease from the module:

```
sudo ipconfig set en10 DHCP
sleep 3
ifconfig en10
```

You should now see a real `inet` address (e.g. `10.32.179.152`) and `status: active`.

Step 6 — Make it your Mac's primary internet connection

By default, macOS may still prefer Wi-Fi/Ethernet for routing. To use the LTE connection as the actual default route:

Check current default route:

```
route -n get default
```

Option A — Reorder network services (keeps Wi-Fi as fallback):

```
networksetup -listnetworkserviceorder
networksetup -ordernetworkservices "USB Ethernet" "Wi-Fi" "Ethernet"
```

(Use the exact service name shown by `listnetworkserviceorder` — it may not be literally "USB Ethernet" on every Mac.)

Option B — Force the default route directly:

```
sudo route change default -interface en10
```

Option C — Simplest test: temporarily disable Wi-Fi

```
networksetup -setairportpower en0 off
```

Turn back on anytime with `networksetup -setairportpower en0 on`.

Step 7 — Verify

```
curl https://ifconfig.me
ping -c 4 -S 10.32.179.152 8.8.8.8
```

(Replace the IP in `-S` with whatever `ifconfig en10` showed you.)

If `curl` returns a public IP different from your Wi-Fi's IP, and the ping succeeds with 0% packet loss, the LTE connection is live and your Mac's traffic is routing through it.

Troubleshooting notes (from real debugging)

Symptom	Cause / Fix
<code>system_profiler</code> shows nothing at all when plugged in	Hardware/power/cable issue — not a driver problem
Device shows in <code>system_profiler</code> but no <code>/dev/tty.*</code> port	Expected — macOS has no driver for Quectel's vendor-specific AT interfaces; this is why we use raw USB (pyusb) instead
<code>en10</code> exists but <code>status: inactive</code> , no IP	Data call not yet bound — module hasn't been told to dial
<code>AT+QICSGP</code> / <code>AT+QIACT</code> return <code>ERROR</code>	Often means a context is already active from boot — check with <code>AT+QIACT?</code> before reconfiguring; don't reconfigure an active context
<code>AT+QNETDEVCTL=1,1,1</code> returns <code>OK</code> but <code>AT+QNETDEVCTL?</code> shows <code>0,0,0,0</code>	Stale internal module state. Fix: reboot the module with <code>AT+CFUN=1,1</code> , wait for USB re-enumeration, then retry the bind sequence fresh
Interface 2 accepts AT commands but some other interfaces (3, 4) time out	Normal — not every one of the 9 USB interfaces is a general AT port; interfaces 2, 7, 8 mirror each other as AT ports, 5/6 are binary diagnostic logs
<code>ping -I en10 ...</code> fails with "invalid multicast interface"	macOS <code>ping</code> doesn't support <code>-I</code> for interface binding like Linux. Use <code>-S <interface_ip></code> instead, or use <code>curl --interface en10</code>

Quick reference — reconnecting after unplug/replug or reboot

Every time the dongle is unplugged and replugged, or the Mac restarts, repeat:

```
sudo python3 ec200u.py          # reboots module + rebinds data call
sudo ipconfig en10 DHCP      # pulls IP (check interface name first)
```

To release the data call cleanly:

```
sudo python3 ec200u.py disconnect
```